

idTech4 Binary Image

Blender 4.x / 5.x .bimage Import Add-on

Version 1.0.0 · Author: Samson & Claude Sonnet

1. Overview

Doom 3 BFG converts every source image it loads into a compiled cache under `generated/images/`, with the extension `.bimage`: a small header plus one or more mip levels of GPU-ready pixel data. This add-on parses that binary format directly and decodes it into an ordinary Blender image datablock.

It is the smallest add-on in the toolchain and the most self-contained. It needs no source directories, no companion add-ons, and no material database — it opens one file and gives you a picture.

Use it when you are working with a BFG installation whose original `.tga` or `.dds` sources are not present, when you want to see exactly what the engine loaded rather than what the artist authored, or when you need to inspect the contents of a particular mip level or cube face.

2. Installation

1. Open Blender and navigate to `Edit > Preferences > Add-ons`.
2. Click `Install...` and select `idTech4_bimage.py`.
3. Tick the checkbox next to `"idTech4 Binary Image (.bimage)"`.
4. Close Preferences. The add-on is now active.

It adds one menu entry: `File > Import > idTech4 Binary Image (.bimage)`.

3. Importing

idTech 4 games do not ship their assets as loose files. Doom 3, Quake 4, Prey and The Dark Mod pack them into `.pk4` archives; Doom 3 BFG packs them into `.resources` files. This add-on does not read inside either. It reads an ordinary directory tree on disk, and nothing else.

IMPORTANT: Extract the game's archives before pointing this add-on at anything inside them. A .pk4 is an ordinary zip archive that any zip tool will open; a .resources file needs a Doom 3 BFG extraction tool. Extract every archive into one folder, preserving the paths inside them, so you end up with the game's real directory structure.

For this add-on that means the BFG .resources archives, which is where the generated/images/ tree lives. Extract them and the .bimage files appear as ordinary files you can browse to.

Use File > Import > idTech4 Binary Image (.bimage) and pick a file. The decoded pixels are put into a new Blender image datablock named after the file, and packed into the .blend so the datablock survives without the source file.

Option	Type	Default	Description
Mip Level	<i>Int</i>	0	Which mip level to decode. 0 is the full-resolution base mip; higher numbers are progressively smaller.
Cube Face	<i>Enum</i>	All 6 (horizontal strip)	For cubemap files, which face to load: Face 0 (+X) through Face 5 (-Z), or all six laid out side by side as one horizontal strip. Ignored for ordinary 2D images.
Force Reload	<i>Bool</i>	Off	Reload even if this exact file, mip and face combination has already been imported. Without it, an already-loaded image is reused and the status line says so.

On success the status line reports the image name, its texture type, its pixel format and colour format, the base dimensions and how many mip levels the file contains — for example, "Loaded bumpmap.bimage: 2D DXT5/NORMAL_DXT5 512x512 (10 mip(s))".

NOTE: Colorspace is chosen automatically from the file's own colour format: Non-Color for normal maps (NORMAL_DXT5) and for the green-alpha packing, sRGB for everything else. That is what makes a decoded normal map usable directly rather than needing to be corrected by hand.

4. What the Format Holds

4.1 Texture Types

Type	Meaning
2D	An ordinary single image.
CUBIC	A cubemap — six faces, in the order +X, -X, +Y, -Y, +Z, -Z. This is where the

Type	Meaning
	Cube Face option applies.
2D_ARRAY	An array of 2D images.
DISABLED	A placeholder with no image data.

4.2 Pixel Formats

A .bimage stores pixels in whichever GPU-ready format the engine chose for that image. The decoder handles:

Format	Notes
RGBA8, XRGB8	Uncompressed 32-bit.
ALPHA, LUM8, L8A8, INT8	Uncompressed single- and dual-channel formats.
RGB565	Uncompressed 16-bit colour.
DXT1, DXT5	Block-compressed, decoded in-process.
BC5, BC7	Block-compressed. These occur in the PS4 build, which reuses format codes 13 and 14 — nominally a mobile-only format and a shadow array on PC — for BC5 and BC7. Since the PC meanings essentially never occur in practice, the two codes are treated as BC5 and BC7 unconditionally. They are decoded by round-tripping through a throwaway temporary DDS file and Blender's own bundled image loader, rather than by a decoder written here.
DEPTH, X16, Y16_X16	Not supported. These are render targets rather than art assets and the import reports them as unsupported rather than producing a meaningless picture.

4.3 Colour Formats

On top of the pixel format, BFG stores several encoder-specific channel layouts, and the decoder undoes each of them:

Colour format	What it is
DEFAULT	Plain RGBA in the natural channel order.
NORMAL_DXT5	A normal map packed into DXT5 so that the two stored channels get the better of DXT5's two encodings; the third is reconstructed.
YCOCG_DXT5, YCOCG_RGBA8	A YCoCg colour-space trick used for diffuse maps, which compresses noticeably better than RGB through DXT5.
GREEN_ALPHA	A mask value packed into the green channel of a DXT1 block.

5. Reference

The format and every decoding path are implemented against the BFG source: `renderer/BinaryImage.cpp`, `DXT/DXTDecoder.cpp` and `RenderProgs_embedded.h`, plus the format constants in `renderer/BinaryImageData.h` and `renderer/ImageOpts.h`.

6. Tips and Troubleshooting

Where the Files Are

Under the game's `generated/images/` tree, mirroring the original texture paths — once the BFG `.resources` archives have been extracted. A `.bimage` is a cache, so it exists only for images the game has actually loaded at least once. If you cannot find a `generated/` folder at all, the assets are still packed.

A Normal Map Looks Washed Out or Wrong

Check the colorspace on the resulting image datablock. It is set automatically from the file's colour format, but if the image is then copied or re-saved by hand that setting can be lost. A normal map must be Non-Color.

"Unsupported format"

DEPTH, X16 and Y16_X16 are render-target formats rather than art, and are refused deliberately. There is nothing useful to see in them.

A Cubemap Came In As One Wide Strip

That is the default. Set Cube Face to a specific face if you want just one, and remember Force Reload — without it, the already-loaded strip is reused rather than the face being decoded.

The Image Is Smaller Than Expected

Mip Level is not 0. Level 0 is the full-resolution base mip.

Re-Importing Does Nothing

The add-on reuses an image datablock already loaded from the same file, mip and face, and says so in the status line. Tick Force Reload to decode it again.